

SQL-ASSIGNMENT-2 MODULE 8

5. Mixed Party + Order Queries

5.1 Shipping Addresses for October 2023 Orders

Business Problem:

Customer Service might need to verify addresses for orders placed or completed in October 2023. This helps ensure shipments are delivered correctly and prevents address-related issues.

```
SELECT oh.ORDER_ID,  
       pcm.PARTY_ID AS "CUSTOMER ID",  
       CONCAT(pr.FIRST_NAME) AS CUSTOMER_NAME,  
       pa.ADDRESS1,  
       pa.CITY,  
       pa.STATE_PROVINCE_GEO_ID,  
       pa.POSTAL_CODE,  
       pa.COUNTRY_GEO_ID,  
       oh.STATUS_ID,  
       oh.ORDER_DATE  
  
FROM order_header oh  
JOIN order_contact_mech ocm  
ON oh.ORDER_ID = ocm.ORDER_ID  
LEFT JOIN postal_address pa  
ON ocm.CONTACT_MECH_ID = pa.CONTACT_MECH_ID  
JOIN party_contact_mech pcm  
ON ocm.CONTACT_MECH_ID = pcm.CONTACT_MECH_ID  
JOIN person pr  
ON pcm.PARTY_ID = pr.PARTY_ID
```

```
WHERE oh.STATUS_ID IN ('ORDER_CREATED','ORDER_COMPLETED')
AND DATE(oh.ORDER_DATE) BETWEEN '2023-10-01' AND '2023-10-31';
```

5.2 Orders from New York

Business Problem:

Companies often want region-specific analysis to plan local marketing, staffing, or promotions in certain areas—here, specifically, New York.

```
SELECT oh.ORDER_ID,
       CONCAT(pr.FIRST_NAME) AS CUSTOMER_NAME,
       pa.ADDRESS1 AS "STREET_ADDRESS",
       pa.CITY,
       pa.STATE_PROVINCE_GEO_ID AS STATE_PROVINCE,
       pa.POSTAL_CODE,
       oh.GRAND_TOTAL,
       oh.STATUS_ID,
       DATE(oh.ORDER_DATE) AS ORDER_DATE

FROM order_header oh
JOIN order_contact_mech ocm
ON oh.ORDER_ID = ocm.ORDER_ID
LEFT JOIN postal_address pa
ON ocm.CONTACT_MECH_ID = pa.CONTACT_MECH_ID
JOIN party_contact_mech pcm
ON ocm.CONTACT_MECH_ID = pcm.CONTACT_MECH_ID
JOIN person pr
ON pcm.PARTY_ID = pr.PARTY_ID
WHERE pa.STATE_PROVINCE_GEO_ID = 'NY';
```

```

1 SELECT oh.ORDER_ID,
2        CONCAT(pr.FIRST_NAME) AS CUSTOMER_NAME,
3        pa.ADDRESS1 AS "STREET_ADDRESS",
4        pa.CITY,
5        pa.STATE_PROVINCE_GEO_ID AS STATE_PROVINCE,
6        pa.POSTAL_CODE,
7        oh.GRAND_TOTAL,
8        oh.STATUS_ID,
9        DATE(oh.ORDER_DATE) AS ORDER_DATE
10
11 FROM order_header oh
12 JOIN order_contact_mech ocm
13 ON oh.ORDER_ID = ocm.ORDER_ID
14 LEFT JOIN postal_address pa
15 ON ocm.CONTACT_MECH_ID = pa.CONTACT_MECH_ID
16 JOIN party_contact_mech pcm
17 ON ocm.CONTACT_MECH_ID = pcm.CONTACT_MECH_ID
18 JOIN person pr
19 ON pcm.PARTY_ID = pr.PARTY_ID
20 WHERE pa.STATE_PROVINCE_GEO_ID = 'NY';

```

RUN LIMIT: 1000 00:00:00.32 SAVE COPY LINK

RESULTS QUERY HISTORY

CREATE CHART DOWNLOAD TO CSV COPY TO CLIPBOARD Filter results

```

SELECT oh.ORDER_ID, CONCAT(pr.FIRST_NAME) AS CUSTOMER_NAME, pa.ADDRESS1 AS "STREET_ADDRESS", pa.CITY, pa.STATE_PROVINCE_GEO_ID AS STATE_PROVINCE, pa.POSTAL_CODE, oh.GRAND_TOTAL, oh.STATUS_ID, DATE(oh.ORDER_DATE) AS O...

```

ORDER_ID	CUSTOMER_NAME	STREET_ADDRESS	CITY	STATE_PROVINCE	POSTAL_CODE	GRAND_TOTAL	STATUS_ID	ORDER_DATE
M100047	Akshansh	New jersey	New jersey	NY	10001	69	ORDER_APPROVED	2026-05-05

1 row

5.3 Top-Selling Product in New York

Business Problem:

Merchandising teams need to identify the best-selling product(s) in a specific region (New York) for targeted restocking or promotions.

```

SELECT oi.PRODUCT_ID,
       p.INTERNAL_NAME,
       SUM(oi.QUANTITY) AS TOTAL_QUANTITY_SOLD,
       pa.CITY,
       pa.STATE_PROVINCE_GEO_ID AS STATE,
       SUM(oi.QUANTITY * oi.UNIT_PRICE) AS REVENUE

FROM order_header oh
JOIN order_item oi
ON oh.ORDER_ID = oi.ORDER_ID
JOIN product p
ON oi.PRODUCT_ID = p.PRODUCT_ID
JOIN order_contact_mech ocm
ON oi.ORDER_ID = ocm.ORDER_ID AND
ocm.CONTACT_MECH_PURPOSE_TYPE_ID='SHIPPING_LOCATION'
LEFT JOIN postal_address pa
ON ocm.CONTACT_MECH_ID = pa.CONTACT_MECH_ID
WHERE pa.STATE_PROVINCE_GEO_ID = "NY"

GROUP BY oi.PRODUCT_ID,

```

p.INTERNAL_NAME,
pa.CITY,
pa.STATE_PROVINCE_GEO_ID

ORDER BY TOTAL_QUANTITY_SOLD DESC, REVENUE DESC;

<pre>SELECT oi.PRODUCT_ID, p.INTERNAL_NAME, SUM(oi.QUANTITY) AS TOTAL_QUANTITY_SOLD, pa.CITY, pa.STATE_PROVINCE_GEO_ID AS STATE, SUM(oi.QUANTITY * oi.UNIT_PRICE) AS REVENUE FROM order_header oh JOIN order_item oi ON oh.ORDER_ID = oi.ORDER_ID JOIN product p ON oi.PRODUCT_ID = p.PRODUCT_ID LEFT JOIN ocr ON ocr.ORDER_ID = oh.ORDER_ID AND ocr.CONTACT_RECH_PURPOSE_TYPE_ID = 'SHIPPING_LOCATION' LEFT JOIN ocr ON ocr.ORDER_ID = oi.ORDER_ID AND ocr.CONTACT_RECH_ID = oi.PRODUCT_ID WHERE pa.STATE_PROVINCE_GEO_ID = 'NY' GROUP BY oi.PRODUCT_ID, p.INTERNAL_NAME, pa.CITY, pa.STATE_PROVINCE_GEO_ID ORDER BY TOTAL_QUANTITY_SOLD DESC, REVENUE DESC;</pre>						
RUN SELECTION LIMIT: 1 000 00:00:00.31						
RESULTS QUERY HISTORY						
CREATE CHART DOWNLOAD TO CSV COPY TO CLIPBOARD Filter results						
PRODUCT_ID	INTERNAL_NAME	TOTAL_QUANTITY_SOLD	CITY	STATE	REVENUE	
M104256	45528481661245	15	Alamagordo	NY	7355.85	
M104285	45528481628477	14	Alamagordo	NY	4879.9	
M103007	MP11-32-Brown	4	Suffern	NY	296	
M104264	45528482382141	3	Suffern	NY	2249.85	
M104260	45528482021693	2	Indore	NY	1399.9	
M104256	45528481661245	1	Testville	NY	885.95	
M104264	45528482382141	1	Brooklyn	NY	749.95	
M104264	45528482382141	1	Indore	NY	749.95	
M103769	sku-managed-1	1	Indore	NY	629.95	
M104266	45528481825085	1	Indore	NY	600	

1	SELECT	oi.PRODUCT_ID,				
2		p.INTERNAL_NAME,				
3		SUM(oi.QUANTITY) AS TOTAL_QUANTITY_SOLD,				
4		pa.CITY,				
5		pa.STATE_PROVINCE_GEO_ID AS STATE,				
6		SUM(oi.QUANTITY * oi.UNIT_PRICE) AS REVENUE				
7						
8		FROM order_header oh				
9		JOIN order_item oi				
10		ON oh.ORDER_ID = oi.ORDER_ID				
11		JOIN product p				
12		ON oi.PRODUCT_ID = p.PRODUCT_ID				
RUN LIMIT: 1 000 00:00:00.31 SAVE COPY LINK						
RESULTS QUERY HISTORY						
CREATE CHART DOWNLOAD TO CSV COPY TO CLIPBOARD Filter results						
SELECT oi.PRODUCT_ID, p.INTERNAL_NAME, SUM(oi.QUANTITY) AS TOTAL_QUANTITY_SOLD, pa.CITY, pa.STATE_P...						19 rows
PRODUCT_ID	INTERNAL_NAME	TOTAL_QUANTITY_SOLD	CITY	STATE	REVENUE	
M104256	45528481661245	15	Alamagordo	NY	7355.85	
M104285	45528481628477	14	Alamagordo	NY	4879.9	
M103007	MP11-32-Brown	4	Suffern	NY	296	
M104264	45528482382141	3	Suffern	NY	2249.85	

7.3 Store-Specific (Facility-Wise) Revenue

Business Problem:

Different physical or online stores (facilities) may have varying levels of performance. The business wants to compare revenue across facilities for sales planning and budgeting.

```
SELECT f.FACILITY_ID,
       f.FACILITY_NAME,
       COUNT(DISTINCT oh.ORDER_ID) AS TOTAL_ORDERS,
       SUM(oi.QUANTITY * oi.UNIT_PRICE) AS TOTAL_REVENUE,
       '2026-04-01 TO 2026-04-30' AS DATE_RANGE

FROM order_header oh
JOIN order_item oi
  ON oh.ORDER_ID = oi.ORDER_ID
JOIN order_item_ship_group oisg
  ON oi.ORDER_ID = oisg.ORDER_ID AND oi.SHIP_GROUP_SEQ_ID =
oisg.SHIP_GROUP_SEQ_ID
JOIN facility f
  ON oisg.FACILITY_ID = f.FACILITY_ID

WHERE DATE(oh.ORDER_DATE) BETWEEN '2026-04-01' AND '2026-04-30'

GROUP BY f.FACILITY_ID,
         f.FACILITY_NAME

ORDER BY TOTAL_REVENUE DESC,
         TOTAL_ORDERS DESC;
```

```

6
7 FROM order_header oh
8 JOIN order_item oi
9 ON oh.ORDER_ID = oi.ORDER_ID
10 JOIN order_item.ship_group oisg
11 ON oi.ORDER_ID = oisg.ORDER_ID AND oi.SHIP_GROUP_SEQ_ID = oisg.SHIP_GROUP_SEQ_ID
12 JOIN facility f
13 ON oisg.FACILITY_ID = f.FACILITY_ID
14
15 WHERE DATE(oh.ORDER_DATE) BETWEEN '2026-04-01' AND '2026-04-30'
16
17 GROUP BY f.FACILITY_ID,
18 f.FACILITY_NAME
19
20 ORDER BY TOTAL_REVENUE DESC,
21 TOTAL_ORDERS DESC;
22

```

RUN LIMIT: 1 000 00:00:00.40 SAVE COPY LINK

RESULTS QUERY HISTORY

CREATE CHART DOWNLOAD TO CSV COPY TO CLIPBOARD Filter results

SELECT f.FACILITY_ID, f.FACILITY_NAME, COUNT(DISTINCT oh.ORDER_ID) AS TOTAL_ORDERS, SUM(oi.QUANTITY * oi.UNIT_PRICE) AS TOTAL_REVENUE, '2026-04-01 TO 2026-04-30' AS DATE_RANGE 3 rows

FACILITY_ID	FACILITY_NAME	TOTAL_ORDERS	TOTAL_REVENUE	DATE_RANGE
NA	Brokering Queue	12	5679.65	2026-04-01 TO 2026-04-30
DEMO_FACILITY	DEMO_FACILITY	1	69	2026-04-01 TO 2026-04-30
GENERAL_OPS_PARKING	General Ops Parking	1	69	2026-04-01 TO 2026-04-30

8. Inventory Management & Transfers

8.1 Lost and Damaged Inventory

Business Problem:

Warehouse managers need to track “shrinkage” such as lost or damaged inventory to reconcile physical vs. system counts.

```

SELECT
iid.INVENTORY_ITEM_ID,
oi.PRODUCT_ID,
oisg.FACILITY_ID,
SUM(ABS(iid.QUANTITY_ON_HAND_DIFF)) AS QUANTITY_LOST_OR_DAMAGED,
iid.REASON_ENUM_ID AS REASON_CODE,
DATE(iid.EFFECTIVE_DATE) AS TRANSACTION_DATE

```

```

FROM inventory_item_detail iid
JOIN order_item oi
  ON iid.ORDER_ID = oi.ORDER_ID
  AND iid.ORDER_ITEM_SEQ_ID = oi.ORDER_ITEM_SEQ_ID
JOIN order_item_ship_group oisg
  ON oi.ORDER_ID = oisg.ORDER_ID
  AND oi.SHIP_GROUP_SEQ_ID = oisg.SHIP_GROUP_SEQ_ID

WHERE iid.REASON_ENUM_ID IN ('VAR_LOST', 'VAR_DAMAGED')

GROUP BY
  iid.INVENTORY_ITEM_ID,
  oi.PRODUCT_ID,
  oisg.FACILITY_ID,
  iid.REASON_ENUM_ID,
  DATE(iid.EFFECTIVE_DATE)

ORDER BY QUANTITY_LOST_OR_DAMAGED DESC;

```

8.2 Low Stock or Out of Stock Items Report

Business Problem:

Avoiding out-of-stock situations is critical. This report flags items that have fallen below a certain reorder threshold or have zero available stock.

```

SELECT
  ii.PRODUCT_ID,
  p.INTERNAL_NAME AS PRODUCT_NAME,
  ii.FACILITY_ID,
  SUM(ii.QUANTITY_ON_HAND_TOTAL) AS QOH,
  SUM(ii.AVAILABLE_TO_PROMISE_TOTAL) AS ATP,
  pf.MINIMUM_STOCK AS REORDER_THRESHOLD,
  CURDATE() AS DATE_CHECKED

FROM inventory_item ii
JOIN product p

```

```
ON ii.PRODUCT_ID = p.PRODUCT_ID
JOIN product_facility pf
ON ii.PRODUCT_ID = pf.PRODUCT_ID AND ii.FACILITY_ID = pf.FACILITY_ID
WHERE ii.STATUS_ID = 'INV_AVAILABLE'

GROUP BY
    ii.PRODUCT_ID,
    p.INTERNAL_NAME,
    ii.FACILITY_ID,
    pf.MINIMUM_STOCK

HAVING SUM(ii.AVAILABLE_TO_PROMISE_TOTAL) <= pf.MINIMUM_STOCK
        OR SUM(ii.AVAILABLE_TO_PROMISE_TOTAL) <= 0

ORDER BY ATP ASC;
```

8.3 Retrieve the Current Facility (Physical or Virtual) of Open Orders

Business Problem:

The business wants to know where open orders are currently assigned, whether in a physical store or a virtual facility (e.g., a distribution center or online fulfillment location).

```
SELECT oh.order_id,
       oh.status_id AS order_status,
       f.facility_id,
       f.facility_name,
       f.facility_type_id

FROM ORDER_HEADER oh
JOIN ORDER_ITEM_SHIP_GROUP oisg
```



```
ON oh.order_id = oisg.order_id  
JOIN FACILITY f  
ON oisg.facility_id = f.facility_id
```

```
WHERE oh.status_id NOT IN (  
'ORDER_COMPLETED',  
'ORDER_CANCELLED',  
'ORDER_REJECTED')
```

```
AND f.facility_id IS NOT NULL
```

```
ORDER BY oh.order_id;
```

1
2
3
4
5
6
7
8
9
10
11
12
13
14
15
16
17
18
19
20

```

SELECT oh.order_id,
oh.status_id AS order_status,
f.facility_id,
f.facility_name,
f.facility_type_id

FROM ORDER_HEADER oh
JOIN ORDER_ITEM_SHIP_GROUP oisg
ON oh.order_id = oisg.order_id
JOIN FACILITY f
ON oisg.facility_id = f.facility_id

WHERE oh.status_id NOT IN (
'ORDER_COMPLETED',
'ORDER_CANCELLED',
'ORDER_REJECTED')

AND f.facility_id IS NOT NULL

ORDER BY oh.order_id;

```

RUN

LIMIT: 1000

00:00:00.17

RESULTS QUERY HISTORY

CREATE CHART

DOWNLOAD TO CSV

COPY TO CLIPBOARD
 Filter results

SELECT oh.order_id, oh.status_id AS order_status, f.facility_id, f.facility_name, f.facility_type_id FROM ORDER_HEADER oh

order_id	order_status	facility_id	facility_name	facility_type_id
1	ORDER_APPROVED	_NA_	Brokering Queue	NA
10	ORDER_APPROVED	_NA_	Brokering Queue	NA
12	ORDER_APPROVED	_NA_	Brokering Queue	NA
12	ORDER_APPROVED	Austin	Austin	RETAIL_STORE
12	ORDER_APPROVED	Austin	Austin	RETAIL_STORE
12	ORDER_APPROVED	Austin	Austin	RETAIL_STORE
12	ORDER_APPROVED	Austin	Austin	RETAIL_STORE
12	ORDER_APPROVED	MAPLEWOOD	Maplewood	RETAIL_STORE
12	ORDER_APPROVED	REJECTED_ITM_PARKING	Rejected Item Parking	NA
13	ORDER_APPROVED	Austin	Austin	RETAIL_STORE
13	ORDER_APPROVED	Austin	Austin	RETAIL_STORE
13	ORDER_APPROVED	Austin	Austin	RETAIL_STORE
13	ORDER_APPROVED	PICKUP_REJECTED	Store Pickup Rejected Queue	NA
13	ORDER_APPROVED	REJECTED_ITM_PARKING	Rejected Item Parking	NA
13	ORDER_APPROVED	REJECTED_ITM_PARKING	Rejected Item Parking	NA
13	ORDER_APPROVED	REJECTED_ITM_PARKING	Rejected Item Parking	NA

8.4 Items Where QOH and ATP Differ

Business Problem:

Sometimes the Quantity on Hand (QOH) doesn't match the Available to Promise (ATP) due to pending orders, reservations, or data discrepancies. This needs review for accurate fulfillment planning.

```
SELECT
  ii.PRODUCT_ID,
  ii.FACILITY_ID,
  SUM(ii.QUANTITY_ON_HAND_TOTAL) AS QOH,
  SUM(ii.AVAILABLE_TO_PROMISE_TOTAL) AS ATP,
  SUM(ii.QUANTITY_ON_HAND_TOTAL) - SUM(ii.AVAILABLE_TO_PROMISE_TOTAL) AS
  DIFFERENCE

FROM INVENTORY_ITEM ii

JOIN PRODUCT p
ON ii.PRODUCT_ID = p.PRODUCT_ID

JOIN FACILITY f
ON ii.FACILITY_ID = f.FACILITY_ID

GROUP BY
  ii.PRODUCT_ID,
  ii.FACILITY_ID

HAVING SUM(ii.QUANTITY_ON_HAND_TOTAL) <>
SUM(ii.AVAILABLE_TO_PROMISE_TOTAL)

ORDER BY ABS(SUM(ii.QUANTITY_ON_HAND_TOTAL) -
SUM(ii.AVAILABLE_TO_PROMISE_TOTAL)) DESC;
```

1
2
3
4
5
6
7
8
9
10
11
12
13
14
15
16
17
18
19
20
21
22

```

SELECT
  ii.PRODUCT_ID,
  ii.FACILITY_ID,
  SUM(ii.QUANTITY_ON_HAND_TOTAL) AS QOH,
  SUM(ii.AVAILABLE_TO_PROMISE_TOTAL) AS ATP,
  SUM(ii.QUANTITY_ON_HAND_TOTAL) - SUM(ii.AVAILABLE_TO_PROMISE_TOTAL) AS DIFFERENCE
FROM INVENTORY_ITEM ii

JOIN PRODUCT p
ON ii.PRODUCT_ID = p.PRODUCT_ID

JOIN FACILITY f
ON ii.FACILITY_ID = f.FACILITY_ID

GROUP BY
  ii.PRODUCT_ID,
  ii.FACILITY_ID

HAVING SUM(ii.QUANTITY_ON_HAND_TOTAL) <> SUM(ii.AVAILABLE_TO_PROMISE_TOTAL)

ORDER BY ABS(SUM(ii.QUANTITY_ON_HAND_TOTAL) - SUM(ii.AVAILABLE_TO_PROMISE_TOTAL)) DESC;

```

RUN

LIMIT: 1000

00:00:23.21

RESULTS QUERY HISTORY

CREATE CHART

DOWNLOAD TO CSV

COPY TO CLIPBOARD
 Filter results

SELECT ii.PRODUCT_ID, ii.FACILITY_ID, SUM(ii.QUANTITY_ON_HAND_TOTAL) AS QOH, SUM(ii.AVAILABLE_TO_PROMISE_TOTAL) AS ATP, SUM(ii.QUANTITY_ON_HAND_TOTAL) - SUM(ii.AVAILABLE_TO_PROMISE_TOTAL) AS DIFFERENCE

PRODUCT_ID	FACILITY_ID	QOH	ATP	DIFFERENCE
M103289	Austin	1000	0	1000
M103640	MAPLEWOOD	100	0	100
M103588	Austin	18	-31	49
M102454	MAPLEWOOD	0	-38	38
M103837	DEMO_FACILITY	0	-10	10
M103588	MAPLEWOOD	-10	-20	10
M103633	Austin	340	336	4
M103993	Austin	-1	-5	4
M102912	MAPLEWOOD	0	-3	3
M103302	Austin	1000	997	3
M102544	MAPLEWOOD	0	-2	2
M103006	Austin	1120	1118	2
M103348	Austin	11077	11075	2
M102912	Austin	-1	-3	2
M103841	Austin	750	748	2
M103590	Austin	100	99	1

8.5 Order Item Current Status Changed Date-Time

Business Problem:

Operations teams need to audit when an order item's status (e.g., from "Pending" to "Shipped") was last changed, for shipment tracking or dispute resolution.

```
SELECT
    os.ORDER_ID,
    os.ORDER_ITEM_SEQ_ID,
    os.STATUS_ID AS CURRENT_STATUS_ID,
    os.STATUS_DATETIME AS STATUS_CHANGE_DATETIME,
    os.STATUS_USER_LOGIN AS CHANGED_BY

FROM ORDER_STATUS os

INNER JOIN (
    SELECT
        ORDER_ID,
        ORDER_ITEM_SEQ_ID,
        MAX(STATUS_DATETIME) AS LATEST_DATETIME
    FROM ORDER_STATUS
    WHERE ORDER_ITEM_SEQ_ID IS NOT NULL
    GROUP BY
        ORDER_ID,
        ORDER_ITEM_SEQ_ID
) latest
ON os.ORDER_ID = latest.ORDER_ID
AND os.ORDER_ITEM_SEQ_ID = latest.ORDER_ITEM_SEQ_ID
AND os.STATUS_DATETIME = latest.LATEST_DATETIME

ORDER BY
    os.ORDER_ID,
    os.ORDER_ITEM_SEQ_ID;
```

1
2
3
4
5
6
7
8
9
10
11
12
13
14
15
16
17
18
19
20
21
22
23
24

```
SELECT
  os.ORDER_ID,
  os.ORDER_ITEM_SEQ_ID,
  os.STATUS_ID AS CURRENT_STATUS_ID,
  os.STATUS_DATETIME AS STATUS_CHANGE_DATETIME,
  os.STATUS_USER_LOGIN AS CHANGED_BY
FROM ORDER_STATUS os

INNER JOIN (
  SELECT
    ORDER_ID,
    ORDER_ITEM_SEQ_ID,
    MAX(STATUS_DATETIME) AS LATEST_DATETIME
  FROM ORDER_STATUS
  WHERE ORDER_ITEM_SEQ_ID IS NOT NULL
  GROUP BY
    ORDER_ID,
    ORDER_ITEM_SEQ_ID
) latest
ON os.ORDER_ID = latest.ORDER_ID
AND os.ORDER_ITEM_SEQ_ID = latest.ORDER_ITEM_SEQ_ID
AND os.STATUS_DATETIME = latest.LATEST_DATETIME
```

RUN

LIMIT: 1 000

00:00:00.32

RESULTS QUERY HISTORY

CREATE CHART

DOWNLOAD TO CSV

COPY TO CLIPBOARD

Filter results

SELECT os.ORDER_ID, os.ORDER_ITEM_SEQ_ID, os.STATUS_ID AS CURRENT_STATUS_ID, os.STATUS_DATETIME AS STATUS_CHANGE_DATETIME, os.STATUS_USER_LOGIN AS				
ORDER_ID	ORDER_ITEM_SEQ_ID	CURRENT_STATUS_ID	STATUS_CHANGE_DATETIME	CHANGED_BY
1	00101	ITEM_APPROVED	2026-05-30T13:54:10.587000	system
10	00101	ITEM_APPROVED	2026-05-26T02:05:03.409000	shrishti.jain
11	00101	ITEM_CANCELLED	2026-05-27T07:02:41.039000	ansh.37
12	00101	ITEM_APPROVED	2026-05-28T03:12:49.302000	kartikey.choudhary
12	00102	ITEM_APPROVED	2026-05-28T03:12:49.302000	kartikey.choudhary
12	00103	ITEM_APPROVED	2026-05-28T03:12:49.302000	kartikey.choudhary
12	00104	ITEM_COMPLETED	2026-05-28T07:01:26.227000	kartikey.choudhary
13	00101	ITEM_APPROVED	2026-05-28T02:18:52.658000	parnika.rangnekar
13	00102	ITEM_APPROVED	2026-05-28T02:18:52.658000	parnika.rangnekar
13	00103	ITEM_APPROVED	2026-05-28T02:18:52.658000	parnika.rangnekar
14	00101	ITEM_APPROVED	2026-05-30T13:54:08.370000	system
14	00102	ITEM_APPROVED	2026-05-30T13:54:08.370000	system
15	00101	ITEM_APPROVED	2026-05-30T13:54:08.499000	system
16	00101	ITEM_CANCELLED	2026-05-27T07:50:04.869000	system
17	00101	ITEM_CREATED	2026-05-27T09:01:03.384000	system
18	00001	ITEM_CREATED	2026-05-28T01:59:06.412000	system
19	00101	ITEM_CANCELLED	2026-05-28T07:22:28.138000	divyansh.bhardwaj
2	00101	ITEM_CANCELLED	2026-05-11T06:58:29.842000	nikunj_bisani
20	00001	ITEM_APPROVED	2026-05-28T01:59:06.412000	system

8.6 Total Orders by Sales Channel

Business Problem:

Marketing and sales teams want to see how many orders come from each channel (e.g., web, mobile app, in-store POS, marketplace) to allocate resources effectively.

```
SELECT
  COALESCE(oh.SALES_CHANNEL_ENUM_ID, 'NOT_SPECIFIED') AS SALES_CHANNEL,
  COALESCE(e.DESCRPTION, 'Unknown') AS CHANNEL_DESCRIPTION,
  COUNT(oh.ORDER_ID) AS TOTAL_ORDERS,
  SUM(oh.GRAND_TOTAL) AS TOTAL_REVENUE,
  DATE_FORMAT(oh.ORDER_DATE, '%Y-%m') AS REPORTING_PERIOD

FROM ORDER_HEADER oh

LEFT JOIN ENUMERATION e
  ON oh.SALES_CHANNEL_ENUM_ID = e.ENUM_ID

WHERE oh.STATUS_ID NOT IN (
  'ORDER_CANCELLED',
  'ORDER_REJECTED'
)

GROUP BY
  oh.SALES_CHANNEL_ENUM_ID,
  e.DESCRPTION,
  DATE_FORMAT(oh.ORDER_DATE, '%Y-%m')

ORDER BY
  REPORTING_PERIOD  DESC,
  TOTAL_ORDERS     DESC;
```

1

2

3

4

5

6

7

8

9

10

11

12

13

14

15

16

17

18

19

20

21

22

```
SELECT
COALESCE(oh.SALES_CHANNEL_ENUM_ID, 'NOT_SPECIFIED') AS SALES_CHANNEL,
COALESCE(e.DESCRPTION, 'Unknown') AS CHANNEL_DESCRIPTION,
COUNT(oh.ORDER_ID) AS TOTAL_ORDERS,
SUM(oh.GRAND_TOTAL) AS TOTAL_REVENUE,
DATE_FORMAT(oh.ORDER_DATE, '%Y-%m') AS REPORTING_PERIOD

FROM ORDER_HEADER oh

LEFT JOIN ENUMERATION e
ON oh.SALES_CHANNEL_ENUM_ID = e.ENUM_ID

WHERE oh.STATUS_ID NOT IN (
'ORDER_CANCELLED',
'ORDER_REJECTED'
)

GROUP BY
oh.SALES_CHANNEL_ENUM_ID,
e.DESCRPTION,
DATE_FORMAT(oh.ORDER_DATE, '%Y-%m')
```

RUN

LIMIT: 1 000

00:00:00.29

RESULTS QUERY HISTORY

CREATE CHART

DOWNLOAD TO CSV

COPY TO CLIPBOARD

Filter results

SELECT COALESCE(oh.SALES_CHANNEL_ENUM_ID, 'NOT_SPECIFIED') AS SALES_CHANNEL, COALESCE(e.DESCRPTION, 'Unknown') AS CHANNEL_DESCR

SALES_CHANNEL	CHANNEL_DESCRIPTION	TOTAL_ORDERS	TOTAL_REVENUE	REPORTING_PERIOD
WEB_SALES_CHANNEL	Web Channel	6	474.22	2026-06
NOT_SPECIFIED	Unknown	1	0	2026-06
WEB_SALES_CHANNEL	Web Channel	167	118829.82	2026-05
NOT_SPECIFIED	Unknown	44	0	2026-05
UNKNWN_SALES_CHANNEL	Unknown Channel	2	0	2026-05
WEB_SALES_CHANNEL	Web Channel	13	5796.05	2026-04
WEB_SALES_CHANNEL	Web Channel	1	0	2026-02
WEB_SALES_CHANNEL	Web Channel	6	2696.85	2026-01
WEB_SALES_CHANNEL	Web Channel	4	137.6	2024-04
WEB_SALES_CHANNEL	Web Channel	10	22713.46	2023-06